



AI Lab Assignment No 1

Submitted by: Hameedullah

Roll No : FA24-BCS-064

Submitted to : Ma'am Zeenat Zulfiqar

Date : 25/4/2026

YOUTUBE VIDEO LINK



GITHUB REPO LINK



TASK NO 1:

Code:

```
from collections import deque

def bfs(graph, start):

    visited = set()

    queue = deque()    # Queue
    queue.append(start)

    bfs_result = []

    while queue:

        node = queue.popleft()

        if node not in visited:

            visited.add(node)
            bfs_result.append(node)

            for neighbor in graph[node]:
                queue.append(neighbor)

    print("BFS Traversal (Queue):")
    print(bfs_result)

def dfs(graph, start):

    visited = set()

    stack = []    # Stack
    stack.append(start)

    dfs_result = []

    while stack:

        node = stack.pop()
```

```
        if node not in visited:

            visited.add(node)
            dfs_result.append(node)

            for neighbor in reversed(graph[node]):
                stack.append(neighbor)

    print("DFS Traversal (Stack):")
    print(dfs_result)

# Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

# Run BFS
bfs(graph, 'A')

# Run DFS
dfs(graph, 'A')
```

output:

```
... 'D': [],  
... 'E': ['F'],  
... 'F': []  
}
```

```
# Run BFS  
bfs(graph, 'A')
```

```
# Run DFS  
dfs(graph, 'A')
```

▼ ... BFS Traversal (Queue):
['A', 'B', 'C', 'D', 'E', 'F']
DFS Traversal (Stack):
['A', 'B', 'D', 'E', 'F', 'C']

TASK NO 2:

```
import time  
import pandas as pd  
import matplotlib.pyplot as plt  
from collections import deque  
  
sizes = [1000, 40000, 80000, 200000, 1000000]  
  
bfs_times = []  
dfs_times = []  
  
# Create linear tree graph  
def create_graph(n):  
  
    graph = {}  
  
    for i in range(n):  
  
        if i < n - 1:  
            graph[i] = [i + 1]  
        else:  
            graph[i] = []
```

```
return graph
```

```
def bfs(graph, start, goal):
```

```
    visited = set()
```

```
    queue = deque()
```

```
    queue.append(start)
```

```
    while queue:
```

```
        node = queue.popleft()
```

```
        if node == goal:
```

```
            return
```

```
        if node not in visited:
```

```
            visited.add(node)
```

```
            for neighbor in graph[node]:
```

```
                queue.append(neighbor)
```

```
def dfs(graph, start, goal):
```

```
    visited = set()
```

```
    stack = []
```

```
    stack.append(start)
```

```
    while stack:
```

```
        node = stack.pop()
```

```
        if node == goal:
```

```
            return
```

```
        if node not in visited:
```

```
            visited.add(node)
```

```

        for neighbor in graph[node]:
            stack.append(neighbor)

# Run for each size
for size in sizes:

    print("Processing size:", size)

    graph = create_graph(size)

    goal = size - 220

    # BFS timing
    start_time = time.time()
    bfs(graph, 0, goal)
    bfs_time = time.time() - start_time

    bfs_times.append(bfs_time)

    # DFS timing
    start_time = time.time()
    dfs(graph, 0, goal)
    dfs_time = time.time() - start_time

    dfs_times.append(dfs_time)

# Create DataFrame
df = pd.DataFrame({
    "Input Size": sizes,
    "BFS Time": bfs_times,
    "DFS Time": dfs_times
})

print("\nResult DataFrame:")
print(df)

x = range(len(sizes))

plt.figure()

```

```
plt.bar(x, bfs_times)
plt.bar(x, dfs_times)

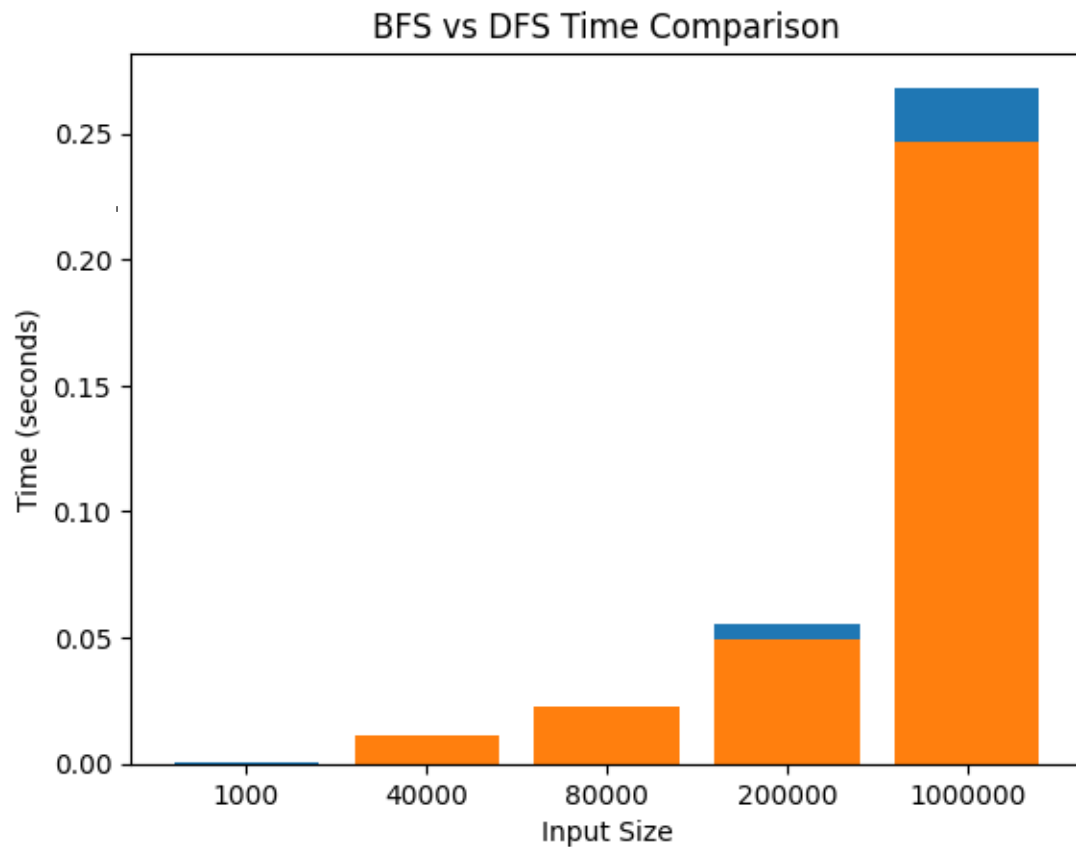
plt.xticks(x, sizes)

plt.xlabel("Input Size")
plt.ylabel("Time (seconds)")
plt.title("BFS vs DFS Time Comparison")

plt.show()
```

Output:

Processing size: 1000
Processing size: 40000
Processing size: 80000
Processing size: 200000
Processing size: 1000000
Result DataFrame:
Input Size BFS Time DFS Time
0 1000 0.000171 0.000163
1 40000 0.009577 0.010838
2 80000 0.021149 0.022686
3 200000 0.055777 0.049571
4 1000000 0.268217 0.246701



TASK NO 3:

```
from collections import deque

# Graph of cities
graph = {

    "Islamabad": ["Rawalpindi", "Lahore", "Peshawar"],

    "Rawalpindi": ["Islamabad", "Peshawar", "Quetta"],

    "Peshawar": ["Islamabad", "Rawalpindi", "Quetta"],

    "Lahore": ["Islamabad", "Multan", "Quetta"],

    "Multan": ["Lahore", "Karachi", "Quetta"],

    "Quetta": ["Rawalpindi", "Peshawar", "Multan", "Karachi"],
```

```

        "Karachi": ["Multan", "Quetta"]
    }

def bfs_shortest_path(graph, start, goal):

    visited = set()

    queue = deque([[start]])

    while queue:

        path = queue.popleft()

        node = path[-1]

        if node == goal:
            return path

        if node not in visited:

            visited.add(node)

            for neighbor in graph[node]:

                new_path = list(path)
                new_path.append(neighbor)

                queue.append(new_path)

    return None

# Run
start_city = "Islamabad"
goal_city = "Karachi"

shortest_path = bfs_shortest_path(graph, start_city, goal_city)

print("Shortest Path from Islamabad to Karachi:")
print(" → ".join(shortest_path))

```

Output:



... Shortest Path from Islamabad to Karachi:
Islamabad → Rawalpindi → Quetta → Karachi

